Dash0 R2 - Call Reference

Project 1: CMS Data Interpolation

The problem

- 80+ markets, CMS-driven product pages, specs hard-coded by editors
- Model year updates → manual hunt across every market
- 4+ driveline variants per car → vague language or duplicated copy

The solution

- Placeholder syntax (`{HIGH:electricRange}`) resolved at request time against live GraphQL API
- `insertData()` runs regex substitution recursively across entire CMS response tree
- Editors write once; spec accuracy maintained automatically

Interesting parts to mention

- Multi-source fallback per field - up to 9 fallback keys depending on certification standard (WLTP, NEDC, CLTC)
- `HIGH` / `LOW` synthetic drivelines computed dynamically - stable vocabulary for editors even when codes change
- Alternative driveline code lookup table (EX90 model year rename: `EN-L` → `DD-L`)
- Debug mode: `?debug=true` wraps substituted values with source markers for editor QA
- Feature-flagged field mapping (LaunchDarkly) for luggage volume priority order

Edge case worth mentioning

- Cargo capacity varies by seating config - dynamic keys (`cargoCapacity5Seats` , `cargoCapacity7Seats`) built at parse time

---

Project 2: Data-Driven Playwright Testing

The problem

- CMS changes happen without a code deploy - traditional E2E tests miss them
- 80+ markets × multiple models → hand-written tests don't scale

Architecture highlights

- Generator script reads `.ct.ts` content type definitions → auto-generates test classes
- Keyed on content type `id` , not class name - survives component renames
- Live CMS data fetched at test time, not fixture snapshots
- New components appear as explicit skipped tests, not silent passes
- Composable tag filtering: market, model, page type - any combination
- Smart link verification: internal links test against current env, external against prod
- WCAG 2AA (axe-core) on every page, every run - not a separate suite
- 900KB HTML size limit enforced in CI
- `isImplemented = false` flag tracks coverage debt explicitly

Numbers: ~32 mapped components, multiple markets (UK, US, zh-CN)

Other Work - Have Ready If Asked

- **CDN cache tagging**: Three-tier `Edge-Cache-Tag` (page type / market / model / market+model) - surgical cache invalidation without global flushes. 7-day page TTL, 30-day asset TTL.
- **`useIdleCallback`**: Below-fold data deferred to browser idle time (500ms deadline, Safari fallback). LaunchDarkly kill switch. Motivation: reducing main thread contention to improve INP scores.
- **Slug → model ID translation**: Consumer slugs (`ex30-electric`) mapped to internal API IDs (`ex30-bev`), including cross-family cases (`ex40-electric` → `xc40` for used car search).

Dash0 Talking Points

Frontend SDK / RSC boundary

- Server-side OTel instrumentation is straightforward (persistent Node process)
- Browser is different: fresh context per page load, RSC blurs the server/client line
- **Key question**: does trace context carry across the RSC/client boundary, or does the browser SDK start a new trace on hydration?
- That boundary is where the hardest performance problems live

Agent0 / autonomous actions

- LaunchDarkly experience: flags that could immediately affect what real users see
- What made it feel safe: full audit trail, two-step activation (set rule ≠ activate rule), easy reversal
- Agent0 is the same trust problem with AI as the actor instead of a person
- Curious about: review step before action, or fire-and-observe?

Deployment-correlated performance regression

- **The real problem**: CWV drops are visible in time series, but attribution to a specific deploy is manual (eyeballing annotations vs. release timestamps)
- What exists elsewhere: Datadog Deployment Tracking, Vercel Speed Insights per-deploy, Sentry Releases - all require setup, none do automatic impact scoring
- **The gap**: both sides of the data exist in Dash0 (CWV from website monitoring, `service.version` on OTel resources) - what's missing is the before/after delta layer
- Post-deploy regression alerting is the simpler wedge: alert if CWV degrades >X% in first N minutes after deploy
- Longer-term: ranked scoring of every deploy across LCP, CLS, INP, error rates
- Engineering questions to raise: deployment event source (CI webhook vs. OTel attribute), window definition (fixed vs. change-point), weighted scoring metric

Dashboard UX / signal vs. noise

- Home Prometheus/Grafana experience: added everything, watched nothing
- Ended up with a handful of panels I actually checked and 1-2 alerts that meant something

- Hardest UI problem isn't showing data - it's helping users identify which of 200 metrics are worth watching
- Curious whether Dash0 is thinking about this at the interface level (beyond just cost reduction)

Alerting

- **Trigger grace period**: condition must hold continuously before firing - single stray request never fires
- **Keep-firing grace period**: alert stays active briefly after condition clears - prevents flapping
- **Enablement conditions**: gate evaluation on minimum request volume - avoids 20% error rate at 5 req/night
- **Two severity tiers**: degraded (Slack) and critical (wake someone up)
- **Pattern that works**: `rate(errors[5m])` threshold + trigger grace period + enablement condition
- **Gap to ask about**: multi-window alerting (short window for speed, long window for confidence) - doable with two PromQL queries but not first-class

---

Questions to Ask

- **Frontend SDK**: "How does `dash0-sdk-web` handle trace context across the RSC/client boundary in Next.js?"
- **Deployment correlation**: "Do you have a way to automatically correlate CWV changes to specific deployments, or is attribution still manual?"
- **Agent0**: "What does the current UX look like for Agent0 actions - review step before it acts, or fire-and-observe?"
- **Alerting maturity**: "Do most users configure grace periods and enablement conditions manually per check, or is there a recommended pattern out of the box?"
- **Dashboard UX**: "What are the hardest UX problems on the product right now - not missing features, but places where the interface makes data harder to use?"
- **Team structure**: "How are frontend decisions made - dedicated frontend team, or full-stack engineers owning slices end to end?"

Don't Claim Depth On

- ClickHouse internals
- Tail sampling implementation
- Perses (know what it is, haven't used it)